

FROM MODEL INTELLIGENCE TO AUTONOMOUS CONTROL

AUTONOMOUS INTELLIGENCE AS A CONTROL ARCHITECTURE

A Self-Driving Race-Car Simulator in 25 Notebook Cells

A rigorous but inspectable demonstration of how perception, state estimation, deterministic planning, strategic language-model judgment, validation, safety governance, execution, telemetry and feedback combine to produce autonomous behavior on a simplified Autódromo Hermanos Rodríguez circuit.

CENTRAL PROPOSITION

Autonomous intelligence is not an LLM. It is a closed-loop architecture of complementary forms of intelligence. The language model can contribute strategic judgment, but sensors observe, estimators construct state, deterministic controllers compute fast actions, a safety governor constrains authority, actuators change the physical system, and telemetry closes the learning and accountability loop. The moving vehicle is the consequence of the architecture as a whole.

Alejandro Reynoso

AI for Financial Practitioners / Autonomous Systems / Control Architecture

October 2026

Pedagogical Research Paper

Educational simulator. The circuit is pedagogically simplified and is not survey-accurate. The vehicle model, sensors, disturbances and control laws are intentionally transparent rather than production-grade. The notebook is designed to explain architecture, not to certify a road-going autonomous-driving system.

Abstract

This paper examines a 25-cell Google Colab notebook that constructs a self-driving race-car simulator as a layered control architecture. The notebook uses a simplified circuit inspired by Autódromo Hermanos Rodríguez and a transparent vehicle model to reveal the informational and authority boundaries often hidden inside the phrase “autonomous intelligence.” Rather than attributing motion to a single intelligent model, the experiment decomposes autonomous behavior into environment representation, sensing, perception packaging, state estimation, physical planning, deterministic control, strategic judgment, validation, safety governance, execution, telemetry and feedback. Each component receives limited information, performs a defined transformation and exercises a bounded form of authority.

The simulator begins by defining track geometry, vehicle constraints and a reproducible physical state. Ideal sensor functions establish what could be observed under perfect conditions; noise and confidence functions then convert those quantities into uncertain measurements. A state estimator constructs a stable operational representation from this evidence. Curvature and estimated grip determine a physically informed target speed, while deterministic lateral and longitudinal controllers generate steering, throttle and braking proposals. This fast loop remains inspectable and functional without generative intelligence, providing both a baseline and a fallback capability.

Claude Haiku 4.5 is introduced only at the slower strategic supervisory layer. It receives a compact situation summary and recommends a bounded policy mode, speed factor, braking margin and overtaking posture. Because generated output is treated as untrusted input, a validation function normalizes types, supplies defaults and clips numerical ranges. A separate deterministic safety governor then evaluates the valid proposal against current curvature, grip, sensor confidence, lateral error and emergency conditions. Only the governed command reaches the vehicle dynamics. Telemetry preserves the estimated state, proposed command, governor intervention and executed action, allowing the causal chain to be reconstructed after every cycle.

The integrated experiment deliberately introduces a dry-to-rain grip transition and a temporary degradation of sensor health across 1,400 simulation steps. Their partial overlap distinguishes physical difficulty from uncertainty about the physical state and tests whether the architecture becomes appropriately conservative without collapsing perception, judgment and execution into one component. The central pedagogical claim is general: autonomous systems become intelligible and governable when designers distinguish what each component may observe, infer, recommend, authorize and execute. The race car therefore serves as a concrete analogy for autonomous financial institutions, where market data, risk estimates, model judgment, compliance controls, transaction authority and audit evidence must likewise remain separate yet coordinated within a closed feedback loop.

COMPANION NOTEBOOK

The complete executable Google Colab notebook is available at [Open the companion self-driving race-car notebook](#).

The Question: What Actually Drives an Autonomous Car?

The notebook begins from a conceptual problem rather than from a coding problem. When a car appears to drive itself, ordinary language compresses many mechanisms into verbs such as “see,” “decide” and “drive.” The notebook reverses that compression. It decomposes autonomous behavior into an explicit sequence of modules, each with a limited informational role and a limited degree of authority.

THE ARCHITECTURAL VIEW

The relevant object of study is not a single intelligent model. It is a recurrent mapping

Environment → Perception → State → Judgment → Governance → Execution → Feedback.

Each arrow is an interface. Each interface can be inspected, tested and governed. Autonomy emerges from repeated circulation through the loop.

The notebook therefore uses the race car as a physical analogy for institutional autonomy. Some computations must be fast, deterministic and mechanically constrained. Others can be slower and more interpretive. A language model is useful where context, synthesis and strategic adaptation matter; it is a poor substitute for the entire control stack.

This layered view is consistent with established autonomous-vehicle research, where successful systems separate sensing, localization, planning, control and vehicle actuation rather than assigning the entire driving task to one undifferentiated component (Thrun et al., 2006; Urmson et al., 2008; Paden et al., 2016; Pendleton et al., 2017). The notebook simplifies those layers for teaching, but retains their most important architectural boundaries.



Figure 1: The complete pedagogical architecture and the 25-cell notebook map.

The phrase “drives the car” is therefore ambiguous unless it is tied to a specific kind of causation. The dynamics model determines how steering, throttle and brake alter motion. The controller selects candidate numerical commands. The strategic supervisor influences the policy under which those commands are calculated. The governor decides whether the proposal is permitted to reach execution. None of these statements is a metaphor: each corresponds to a separately inspectable function and data structure in the notebook. Asking which single component drives the car is like asking which single organ enables a person to run. The useful answer describes the coordinated system and identifies where each contribution begins and ends.

This decomposition also prevents a common category error in discussions of artificial intelligence. A component may be intelligent in the sense that it recognizes patterns, integrates context or recommends a course of action, yet still possess no direct operational authority. Conversely, a simple deterministic function may possess decisive authority even though nobody would describe it as broadly intelligent. The safety governor illustrates the distinction. Its logic is comparatively compact, but because it sits immediately before execution it can veto or reshape every proposed command. Architectural position, not rhetorical sophistication, determines practical power.

The closed loop adds another important idea: no decision is final in isolation. A command changes the vehicle, the changed vehicle generates a new observation, and the next cycle revises the estimate and action. Competence is therefore measured across trajectories rather than single responses. A plausible recommendation that arrives too late, ignores uncertainty or cannot be converted into a valid command contributes little to successful autonomy. By contrast, modest components can produce robust behavior when their interfaces, cadences and permissions are well designed.

For researchers, the notebook offers an experimental vocabulary for evaluating such systems. One can ask whether perception preserves uncertainty, whether state estimation filters noise without hiding hazards, whether control remains stable, whether validation rejects malformed output, whether governance enforces invariants and whether telemetry supports reconstruction. These questions are more precise than asking whether “the AI” performed well. They turn an impressive demonstration into a falsifiable architecture whose modules can be replaced, stressed and compared.

The central answer is consequently plural. The car is driven physically by its actuators and dynamics, tactically by deterministic control, strategically by bounded policy judgment, and institutionally by the rules that determine which proposals are allowed to become actions. Autonomous behavior arises only when those layers circulate information quickly enough and preserve the intended hierarchy of authority. That is the conceptual foundation for every later cell.

Cells 1–5: Building the World Before Building Intelligence

Configuration and reproducibility

Cells 1 and 2 establish the computational environment and the physical constants. The notebook fixes a random seed and defines quantities such as the integration time step, wheelbase, maximum steering, acceleration and braking limits. This is more than housekeeping. Reproducibility is a prerequisite for comparing architectures: if the environment changes randomly between runs, one cannot know whether a different outcome came from the architecture or from the experiment.

Track geometry as a reference system

Cells 3 and 4 build a smooth, closed teaching circuit inspired by the rhythm of Autódromo Hermanos Rodríguez. The notebook explicitly states that the geometry is not survey-accurate. The track is represented as an ordered centerline with segment lengths, cumulative distance, heading and curvature. These quantities become the common coordinate system used later by sensors, state estimation and control.

The key point is that geometry is not yet “intelligence.” It is structured context. Curvature provides a forward-looking physical constraint: tighter curvature reduces the speed that can be carried safely for a given grip level. Heading provides a local direction of travel. Nearest-point calculations allow the system to measure lateral displacement from the reference path.

Vehicle state

Cell 5 defines the state of the car as a compact object containing position, yaw, speed, steering, grip, lap and step. This separation is conceptually important. The physical state exists whether or not the sensors observe it correctly. Later cells will create measurements of this state and estimates of it; those are not the same thing.

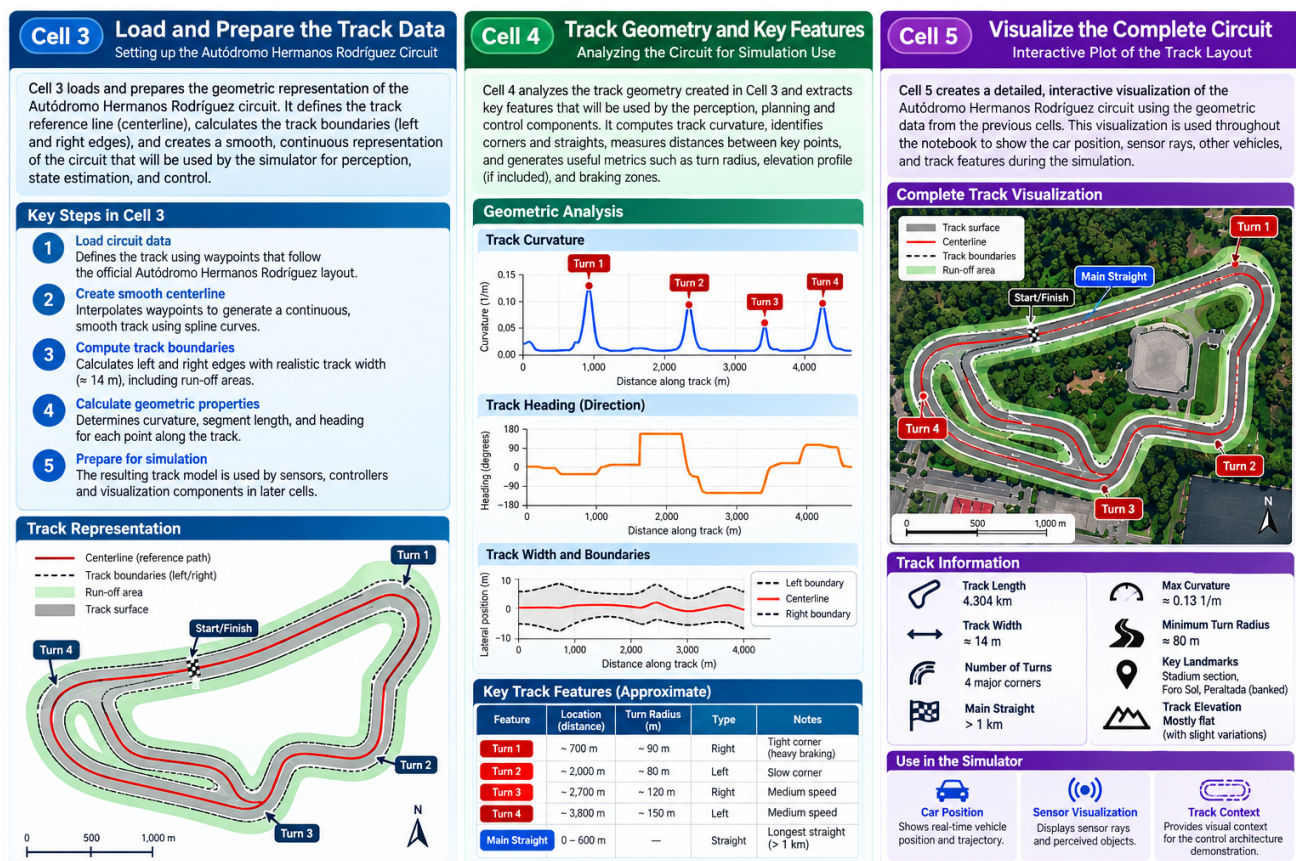


Figure 2: Cells 3–5: track construction, geometric analysis and the state representation used by the simulator.

Pedagogical principle. Before asking an AI system to reason, define the world in which it operates: state variables, coordinate systems, physical limits, admissible actions and measurable quantities. Without that ontology, “reasoning” has no stable object.

The configuration layer also establishes units and conventions, which are easy to overlook but essential to dependable computation. A steering limit is meaningless unless its angular unit is known; a speed limit cannot be compared with a measured velocity if one is expressed in metres per second and the other in kilometres per hour. The notebook keeps these conventions close to the constants so that later formulas remain readable. This is a small example of a broader engineering rule: semantic assumptions should be encoded near the values they govern rather than left as undocumented knowledge.

The track representation converts a visual shape into quantities that algorithms can use. Sampling the centerline creates a finite sequence of reference points. Differences between neighbouring points provide segment direction and length; accumulated lengths create a progress coordinate; changes in heading provide an approximation to curvature. These derived fields allow different modules to speak a shared language. A sensor can report the nearest track index, a planner can inspect curvature ahead, and telemetry can associate a behavior with a location on the circuit. The reference frame is therefore an integration device as much as a geometric construction.

The vehicle state performs a similar integrating function across time. Position and yaw describe pose, speed describes longitudinal motion, steering records the current control configuration, and grip links the vehicle to environmental conditions. Lap and step indices supply temporal context. Keeping these values in an explicit state object avoids hidden global assumptions and makes state transitions inspectable. Given a prior state and an approved command, the dynamics function can produce a next state; given the same inputs and seed, another researcher can reproduce the transition.

There is also a deliberate asymmetry between the simulated ground truth and what the controller is permitted to know. The simulator must retain the true state in order to evolve the world, but the autonomy stack should act through measurements and estimates. If downstream control were allowed to read every ground-truth variable directly, sensor uncertainty would become decorative rather than consequential. By separating physical state from observation at the beginning, the notebook prepares a valid test of perception and estimation in later cells.

Together, Cells 1–5 define the ontology, geometry, constraints and memory of the experiment. They do not make the vehicle autonomous, but they determine whether later claims about autonomy can be interpreted. A reproducible world provides stable causes; a common reference system provides shared meaning; and an explicit state provides continuity. Building these foundations first makes each later intelligence component replaceable without changing what the experiment is fundamentally about.

Cells 6–8: Reality, Sensors and Uncertainty

Cell 6: a transparent bicycle-model dynamics layer

Cell 6 implements simplified vehicle dynamics. Steering, throttle and brake are clipped to physical limits. Longitudinal acceleration combines propulsion, braking, grip and a speed-dependent drag term. Yaw evolves through a bicycle-model relationship between speed, wheelbase and steering angle. Position is then advanced through the time step.

The model is deliberately modest. Its purpose is to preserve causality: commands change motion; motion changes state; state generates new observations. A more realistic simulator could replace this dynamics function without changing the architecture around it.

Cell 7: ideal sensors

The ideal sensor suite computes track index, lateral error, heading error, current curvature, curvature ahead and true grip. This creates a clean conceptual benchmark: what would the system observe if sensing were perfect? The sensor layer does not decide what the car should do. It measures aspects of the world.

Cell 8: noisy sensors and confidence

Cell 8 then destroys the fiction of perfect observation. A sensor-health parameter controls measurement noise, and the output includes a confidence measure. This introduces a second-order variable: the architecture represents not only an observation but also something about the reliability of that observation.

UNCERTAINTY IS PART OF THE STATE OF KNOWLEDGE

A robust autonomous architecture should not treat every measurement as equally trustworthy. Confidence affects downstream authority. In this notebook, low confidence eventually causes the safety governor to reduce throttle and increase braking. The system therefore reacts not only to the estimated road state but also to uncertainty about that estimate.

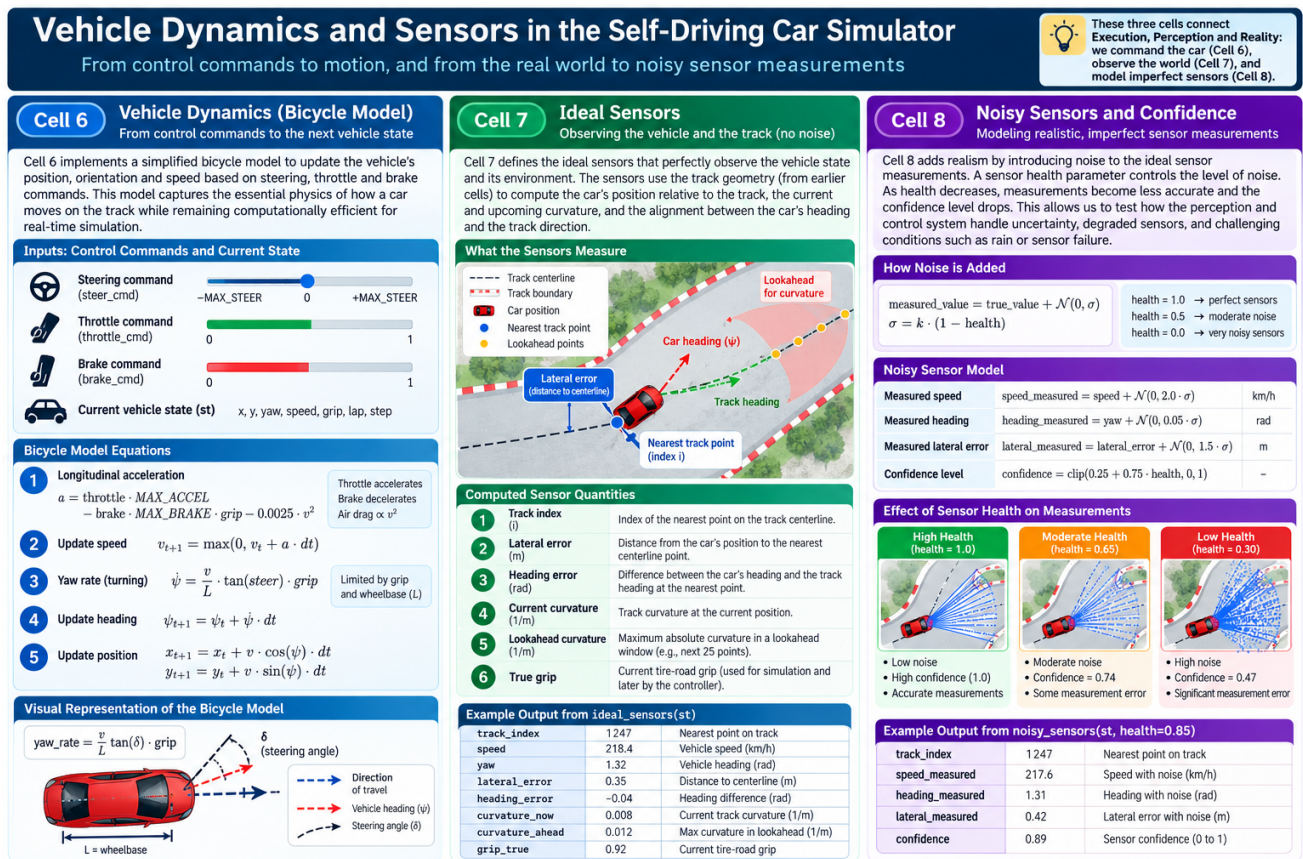


Figure 3: Cells 6–8: physical dynamics, ideal observation and the deliberate introduction of sensor noise and confidence.

The bicycle model is especially useful pedagogically because it is simple enough to inspect while retaining the coupling that matters for the architecture. Steering changes yaw rather than translating the car sideways instantaneously. Forward speed affects how quickly yaw develops, and yaw determines the direction in which position advances. Throttle and brake therefore influence more than travel time: by changing speed, they alter how aggressively a steering input changes the trajectory. This coupling gives the governor a meaningful reason to reduce speed when grip, confidence or lateral position becomes unsafe (Rajamani, 2012).

Ideal sensing and noisy sensing are separated so that uncertainty is introduced deliberately rather than accidentally. The ideal sensor function defines the quantity the architecture would like to know. The noise function then models the imperfect channel through which that quantity becomes available. Because the two stages are distinct, a researcher can inspect bias, variance and confidence independently from track geometry or vehicle dynamics. Alternative noise distributions, delayed measurements or failed channels could be inserted without rewriting the rest of the system.

Confidence is not presented as a mystical probability of correctness. It is an operational signal summarizing sensor health for downstream policy. Its value matters because the architecture maps it to behavior. When confidence falls, the safe response is not necessarily to guess more forcefully; it is to reduce the vehicle's kinetic exposure while better information is unavailable. The later governor implements this principle by restricting throttle and adding brake. In that sense, uncertainty becomes a control input.

This design also shows why redundancy and diagnostic information matter in real autonomous systems. A numerical measurement without provenance or health metadata can appear precise even when its generating channel is degraded. By carrying sensor health alongside the measured fields, the packet tells downstream modules both what was observed and how cautiously it should be interpreted. More advanced implementations might represent separate confidence values for position, speed, heading and grip, but the scalar signal used here is sufficient to expose the architectural idea.

Cells 6–8 complete the causal bridge from action to knowledge. The dynamics layer turns an authorized command into a changed physical state. The ideal sensor layer identifies observable features of that state. The noisy sensor layer turns those features into fallible evidence. The next stages must reason from that evidence rather than from omniscient access. This boundary makes the subsequent estimator and governor necessary, and it ensures that the integrated experiment tests adaptation under uncertainty instead of merely animating a predetermined path.

Cells 9–14: From Raw Measurements to Fast Driving Commands

Perception packet and state estimation

Cell 9 packages noisy measurements into a structured perception packet. This is an interface contract: downstream components receive named fields rather than unstructured sensor output. Cell 10 then maintains a running estimate of important state variables. The estimator smooths measured speed and updates its estimate of grip, while preserving curvature, lateral error and confidence. The conceptual separation between physical state, noisy observation and recursively updated estimate follows a foundational pattern in control and estimation theory (Kalman, 1960).

This creates a critical distinction:

$$\text{measurement} \neq \text{estimated state} \neq \text{desired action}.$$

The system does not control directly from raw sensor noise. It first constructs a more stable representation of the world.

Planning from curvature and grip

Cell 11 translates physical context into a safe target speed. In simplified form, the lateral-acceleration relationship is

$$v_{\text{safe}} \approx \sqrt{\frac{a_y}{|\kappa|}},$$

where κ is curvature and the available lateral acceleration depends on estimated grip and a margin. The notebook clips the resulting target into pedagogically chosen bounds. The important idea is that planning is constrained by physics before any language model enters the system.

Fast lateral and longitudinal control

Cells 12 and 13 implement deterministic steering and speed control. The lateral controller uses track position and heading error to steer toward the reference path. The longitudinal controller compares target speed with estimated speed and maps the error into throttle or brake. These functions are fast, local and transparent.

Cell 14: tactical proposal

Cell 14 combines planning and control under a policy. The policy can alter the speed factor and braking margin, but the tactical controller remains deterministic. It outputs steering, throttle, brake, target speed and supporting fields. At this point the system has a proposed action, not yet an authorized action.

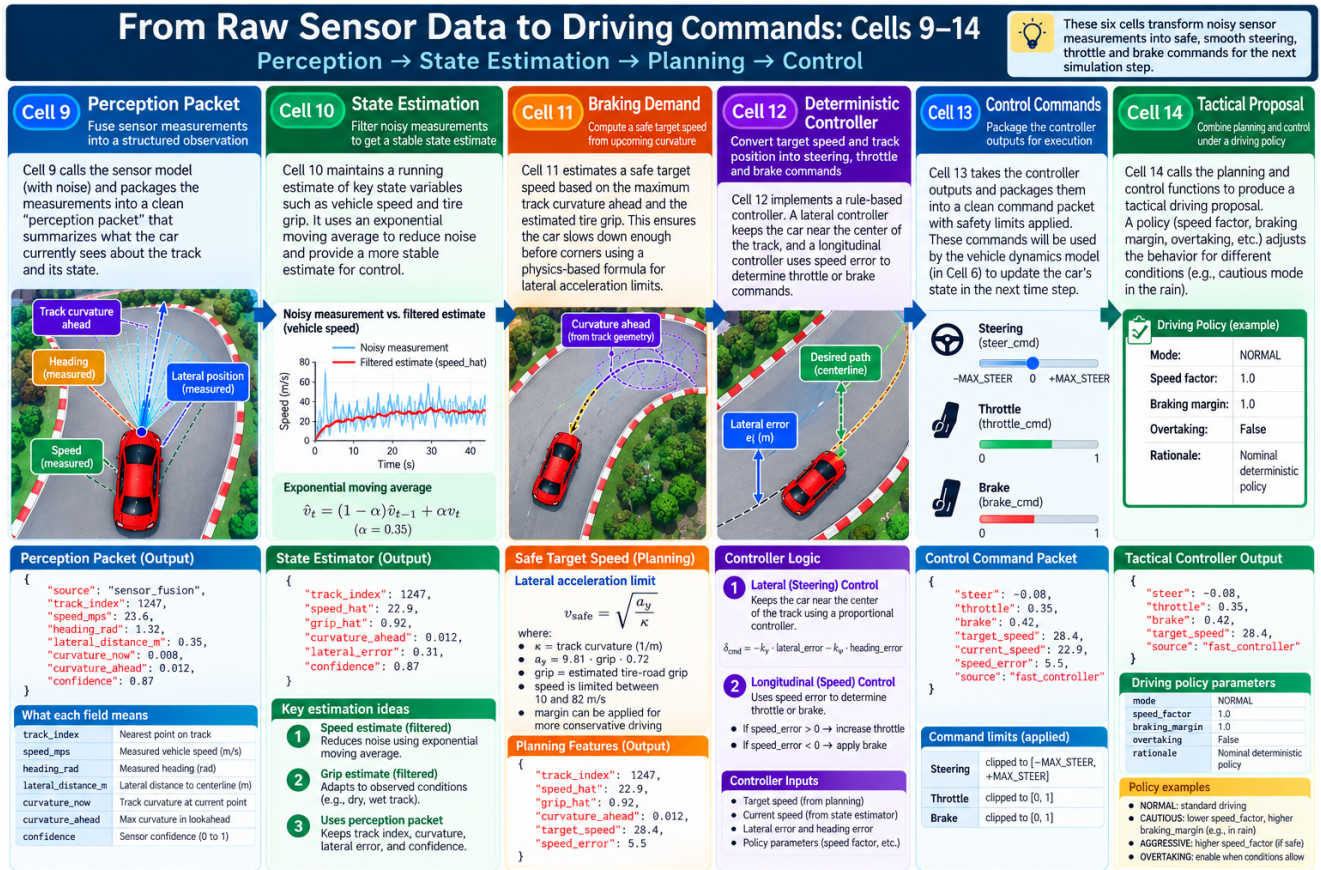


Figure 4: Cells 9–14: the transformation from noisy perception through estimation and planning into a deterministic tactical proposal.

WHY THIS LAYER SHOULD REMAIN DETERMINISTIC

The fast loop has a narrow objective and a short time horizon. It benefits from bounded mathematics, clipping and direct observability. Strategic intelligence can influence parameters of this loop, but it does not need to replace the loop. This is a general architecture pattern: use generative intelligence where interpretation is valuable and deterministic computation where precision, speed and invariants dominate.

Packaging perception before estimation has a practical testing advantage. A recorded packet can be replayed into different estimators, and a synthetic packet can be created to test rare situations without running the full simulation. Named fields also make missing data and schema changes visible. This is important because modularity depends less on drawing boxes than on maintaining clear contracts between them. If a controller silently reads simulator internals that are absent from the packet, the apparent modular architecture would be misleading.

The estimator trades responsiveness against stability. Heavy smoothing can suppress harmless noise but delay recognition of a sudden loss of grip; weak smoothing can follow changes quickly but transmit measurement jitter into the controller. The notebook uses a transparent compromise so the reader can see this trade-off rather than treating state estimation as a black box. Its output is not claimed to be the true state. It is the best operational representation available to the next stage under the chosen assumptions.

Planning then converts descriptive state into a normative target: given curvature, grip and margin, how fast should the vehicle travel? The square-root relationship shows why the answer is nonlinear. Doubling available lateral acceleration does not double safe speed, and a sharp rise in curvature can require substantial deceleration. Looking ahead is crucial because braking must begin before the vehicle reaches the demanding segment. Even in a simplified simulator, planning is therefore predictive rather than purely reactive.

The lateral and longitudinal controllers divide the local action problem into understandable parts, but their outputs remain coupled through motion. Steering reduces geometric error, while throttle and brake regulate the speed at which that correction unfolds. Cell 14 coordinates them under the current policy and publishes both the commands and the target that motivated them. Including supporting fields is valuable for audit: an analyst can distinguish a strong brake command caused by a low target speed from one later imposed by the governor.

At the end of Cell 14, the architecture has completed perception, estimation, physical planning and fast control without requiring a language model. This baseline matters. It demonstrates which capabilities arise from deterministic

computation and creates a safe fallback if strategic supervision is unavailable. The next layer can improve contextual adaptation by changing bounded policy parameters, but basic path following and speed control do not depend on unconstrained generated text.

Cells 15–16: Strategic Judgment Without Direct Control

Cell 15: Claude Haiku 4.5 as supervisor

Cell 15 connects the model `claude-haiku-4-5-20251001`, documented by Anthropic as a small, fast hybrid-reasoning model (Anthropic, 2025). The supervisor receives a compact summary of the current driving situation rather than raw actuator authority. Its role is analogous to a race engineer: interpret conditions and recommend a high-level driving policy. The policy vocabulary is intentionally small: `NORMAL`, `CAUTIOUS` or `EMERGENCY`, together with a speed factor, braking margin, overtaking flag and short rationale.

The supervisor is therefore strategically expressive but operationally bounded. It can recommend that the system become more cautious; it cannot directly write an arbitrary steering angle to the vehicle.

Cell 16: recommendation validation

The output of a language model is treated as untrusted structured input. Cell 16 normalizes the policy. Unknown modes are replaced with `CAUTIOUS`; the speed factor is clipped to $[0.45, 1.05]$; braking margin is clipped to $[1.0, 1.6]$; overtaking is coerced to Boolean; and the rationale is truncated.

A LANGUAGE MODEL OUTPUT IS NOT AN ACTUATOR COMMAND

Validation is a boundary between semantic judgment and machine authority. The supervisor may be persuasive, but persuasion does not enlarge its permissions. The deterministic system decides which values are admissible.

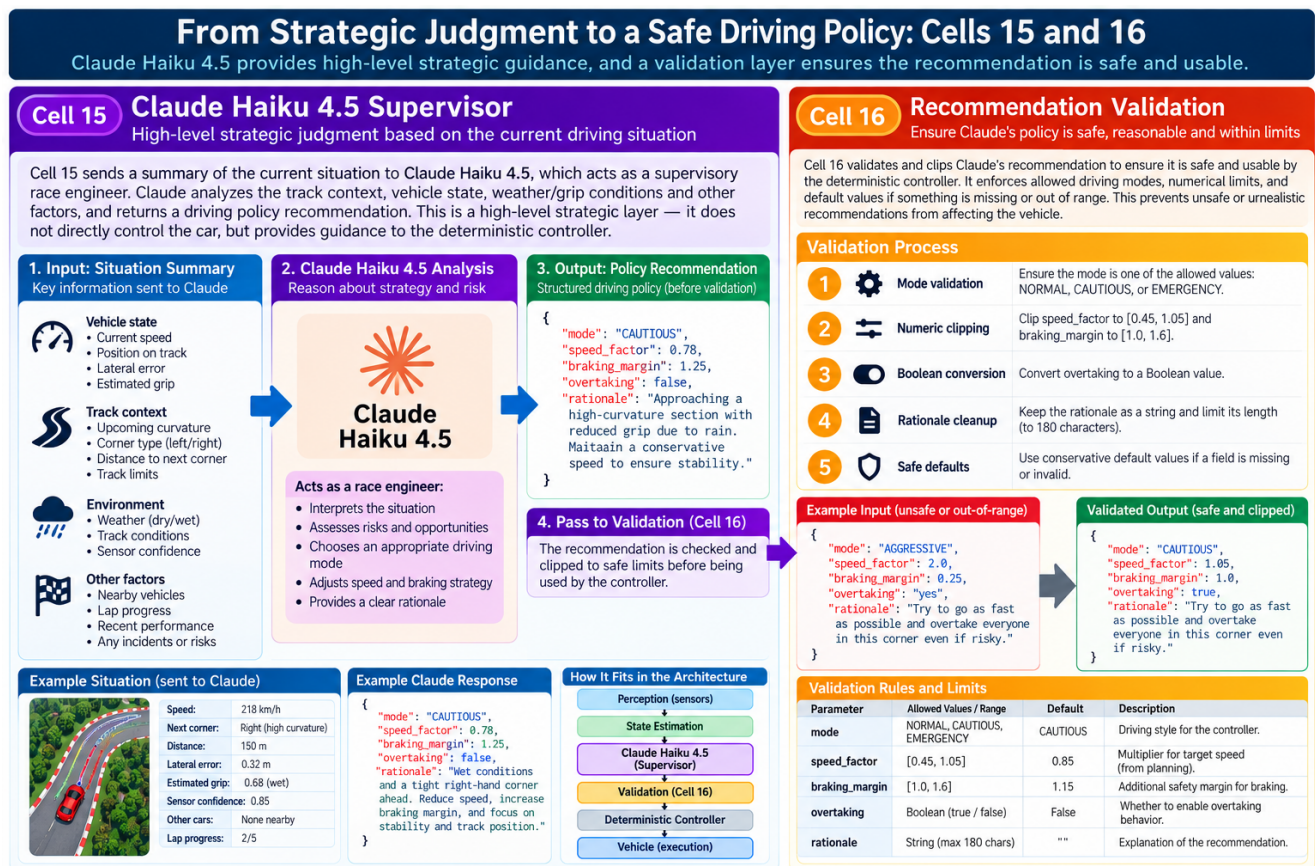


Figure 5: Cells 15–16: strategic language-model judgment followed by deterministic validation and normalization.

The supervisor's input is intentionally summarized. Sending every raw sample would enlarge the context, obscure the relevant features and tempt the model to perform calculations already handled more reliably elsewhere. A compact situation report can instead state estimated grip, confidence, lateral error, speed, anticipated curvature and recent governor behavior. This is a form of information governance: the architecture selects the evidence required for the model's role while withholding irrelevant implementation detail and direct actuator access.

The restricted output vocabulary serves the same purpose on the other side of the interface. Modes and numerical factors translate open-ended judgment into a small policy space that deterministic code can interpret. The model

can explain why rain, poor confidence or repeated interventions justify caution, but its operational influence is confined to parameters with explicit ranges. Natural language remains useful for rationale and human inspection; structured fields carry machine authority.

Validation protects against several distinct failure classes. A response may be malformed, omit a field, use an unexpected label, encode a number as text or suggest a value outside the designed envelope. Normalization supplies defaults and converts types where possible. Clipping then limits valid numbers to admissible ranges. These checks do not establish that the recommendation is strategically wise. They establish that it can safely enter the next computational stage without breaking its contract.

That distinction between validity and wisdom is essential. A speed factor of 1.05 may be syntactically valid and inside the permitted interval, yet still be inappropriate during rain or sensor degradation. The safety governor must evaluate the policy in relation to the current estimated state. Conversely, a cautious recommendation should not be discarded merely because its prose rationale is unusual. Separating validation from governance prevents one mechanism from carrying two conceptually different responsibilities.

The slower supervisory cadence also limits both computational cost and behavioral volatility. Policy need not be regenerated at every integration step because strategic conditions change more slowly than steering error. Holding a validated policy across several fast cycles gives deterministic control a stable objective. When the supervisor is called again, telemetry and the updated situation summary provide feedback about the consequences of the prior recommendation.

Cells 15–16 therefore demonstrate a disciplined use of generative intelligence. The model contributes synthesis where fixed rules may be cumbersome, but it participates through a narrow interface, bounded vocabulary, explicit defaults and deterministic range checks. If the API is unavailable, a local default policy can preserve the fast control loop. If the output is anomalous, validation can degrade safely. Strategic intelligence enhances the architecture without becoming a single point of unchecked operational authority. This emphasis on mapping, measuring, managing and governing model risk is consistent with the lifecycle orientation of the NIST AI Risk Management Framework (National Institute of Standards and Technology, 2023).

Cells 17–20: Governance, Execution, Feedback and Evidence

Cell 17: the deterministic safety governor

The safety governor is the decisive authority boundary. It receives the tactical proposal and can accept or modify it. Four checks are explicit in the notebook. First, target speed cannot exceed the physical envelope implied by curvature and estimated grip. Second, perception confidence below 0.60 limits throttle and imposes braking. Third, a lateral error above 12 triggers strong corrective braking. Fourth, an **EMERGENCY** policy forces zero throttle and at least 0.75 braking.

The governor returns both the modified command and an auditable verdict with reasons. Safety is therefore represented as an enforceable invariant, not merely as a natural-language aspiration. The pattern resembles runtime-assurance and Simplex architectures in which an advanced controller remains subordinate to a dependable safety mechanism (Seto et al., 1998).

Cell 18: execution

Execution is intentionally thin. The approved steering, throttle and brake are passed to the vehicle dynamics function. This makes authority explicit: only the governed command reaches the physical plant.

Cell 19: the closed loop

Cell 19 assembles one complete control cycle:

perceive → estimate → propose → govern → execute.

The function returns both the new vehicle state and a packet containing perception, estimate, policy, proposal, governor verdict and execution. The packet is a trace of how the action was produced.

Cell 20: telemetry

Telemetry converts the control packet into a row suitable for analysis and visualization. It records step, position, speed, grip, confidence, mode, target speed, throttle, brake, steering and governor verdict. This is the architecture's evidentiary layer: a post-run analyst can distinguish what the system believed, what it proposed, what was modified and what was actually executed.

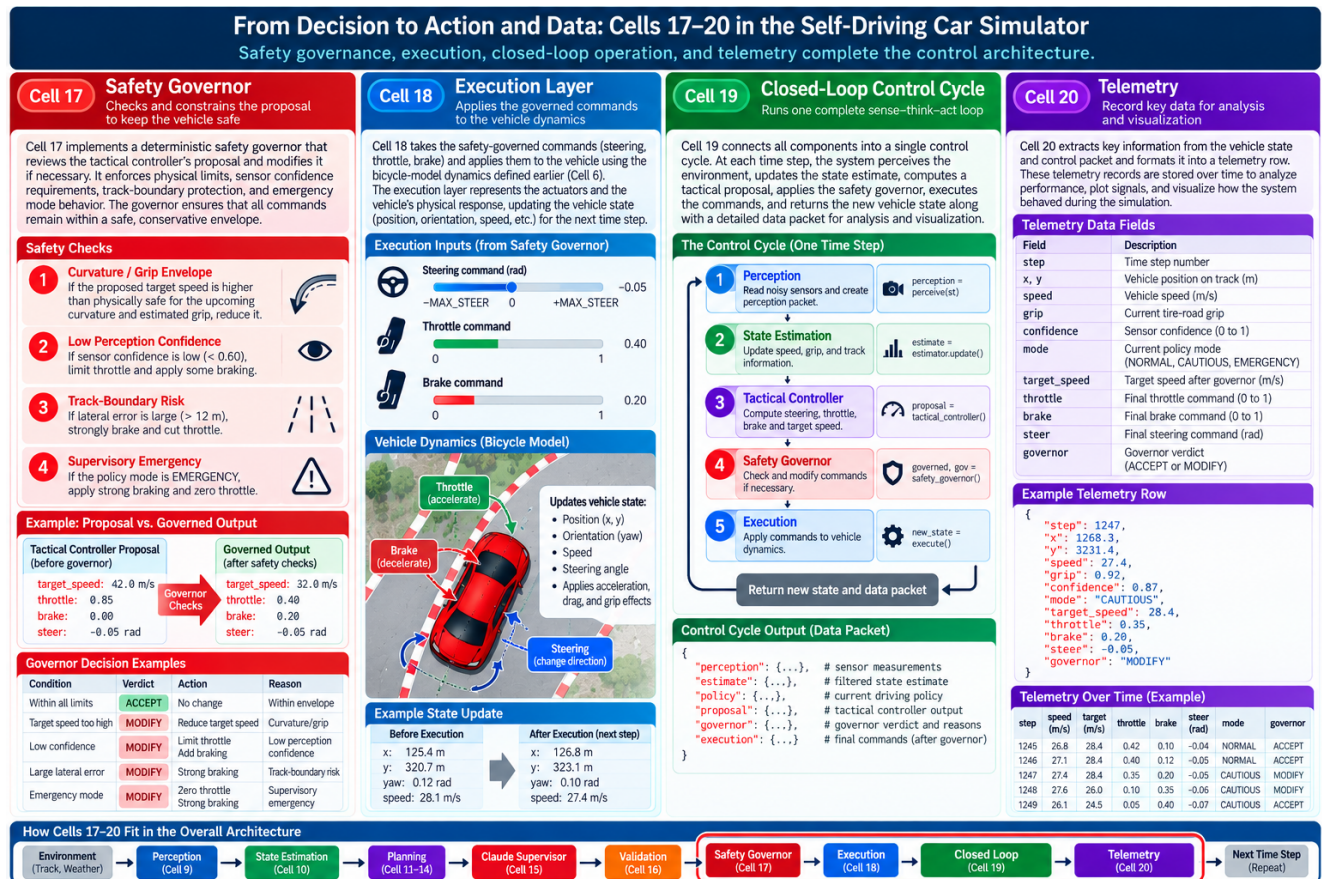


Figure 6: Cells 17–20: deterministic safety governance, actuator execution, the closed-loop control cycle and telemetry.

THE GOVERNOR CHANGES THE MEANING OF AUTONOMY

Autonomy does not require unconstrained action. In this architecture, autonomy means that the system can repeatedly sense, estimate, reason and act without human intervention while remaining inside explicit machine-enforced boundaries. The governor is therefore not external to autonomy; it is part of the autonomous design.

The four governor checks illustrate complementary kinds of constraint. The curvature-and-grip rule protects a physical envelope. The confidence rule responds to epistemic risk: danger created by not knowing the state reliably. The lateral-error rule reacts to an observed departure from the desired corridor. The emergency-mode rule implements a high-level operating posture. Combining these checks demonstrates that safety is rarely reducible to one threshold. It is a composition of constraints grounded in physics, information quality, geometric position and policy.

Order and traceability matter when constraints interact. A proposal could simultaneously exceed the safe-speed envelope and arrive during low confidence. The governor must produce a command that satisfies all active restrictions, and its verdict should retain every relevant reason rather than only the last modification. This enables a later analyst to distinguish frequent interventions caused by aggressive planning from interventions caused by degraded sensing. The command is an operational result; the reasons are control evidence.

Execution remains thin by design because it marks the end of deliberation. Once a command has passed through governance, the execution layer should reinterpret strategy or silently add new policy. It translates the approved values into the call expected by the dynamics model. In a physical platform, this boundary would correspond to actuator interfaces, timing requirements and hardware checks. Keeping it explicit makes clear where digital recommendation becomes physical consequence.

The control-cycle packet then preserves the difference between proposal and action. This is indispensable whenever a governor can intervene. Logging only the executed brake value would reveal what happened but not whether the tactical controller requested it or the safety layer imposed it. Logging only the proposal would obscure the command that actually changed the vehicle. Recording both supports causal reconstruction and permits quantitative measures such as intervention frequency, intervention magnitude and the conditions under which overrides occur.

Telemetry also supports development rather than merely post-event accountability. Researchers can plot target speed against actual speed, confidence against braking, lateral error against steering and proposed commands against

governed commands. Patterns in those comparisons reveal tuning problems and interface failures. A controller that repeatedly pushes against the governor may technically remain safe while indicating that planning assumptions are misaligned. Thus, audit data becomes feedback for architectural improvement.

Cells 17–20 complete the chain of authority. Earlier modules observe, infer and recommend; the governor determines admissibility in context; execution changes the plant; and telemetry records the transition. The design makes a strong claim: trustworthy autonomy depends not only on producing good actions but on making the transformation from evidence to proposal to authorized action visible. Governance and evidence are therefore active computational layers, not documentation added after the system has already acted.

Cells 21–25: Disturbance, Adaptation and the Integrated Experiment

Cell 21: rain as an exogenous disturbance

The grip schedule creates three regimes: dry conditions with grip 1.00 until step 500; rain with grip 0.62 until step 1100; and partial recovery to 0.78 thereafter. The disturbance is external to the controller. The architecture must detect its consequences and adapt.

Cell 22: sensor degradation

Sensor health remains at 1.00 until step 750, falls sharply to 0.42 until step 1050, and then recovers to 0.90. Importantly, the rain and sensor disturbances overlap only partly. This lets the experiment distinguish a difficult physical environment from uncertainty about the environment.

Cell 23: architecture runner

The runner initializes the vehicle and estimator, applies disturbances at each step, optionally calls Claude at a slower interval, executes the fast control cycle, logs telemetry and keeps the demonstration running with a pedagogical reset if the car departs badly from the track. The language model therefore operates on a slower supervisory cadence than the deterministic control loop.

Cell 24: live signal-flow visualization

The renderer turns architectural abstraction into an observable process. One panel shows the vehicle moving on the track; the other displays the current flow through environment, perception, state estimation, judgment, safety governor, execution and feedback. The dashboard is pedagogically important because it reveals that the car's motion is downstream of a chain of messages and transformations.

Cell 25: integrated classroom experiment

The final cell runs 1,400 simulation steps with disturbances and, when enabled, Claude supervision. It displays the animation and computes summary statistics such as mean speed, maximum speed, governor interventions and low-confidence cycles. The notebook's own interpretation is explicit: the moving vehicle is the consequence of the architecture; the LLM is only one bounded component inside it.

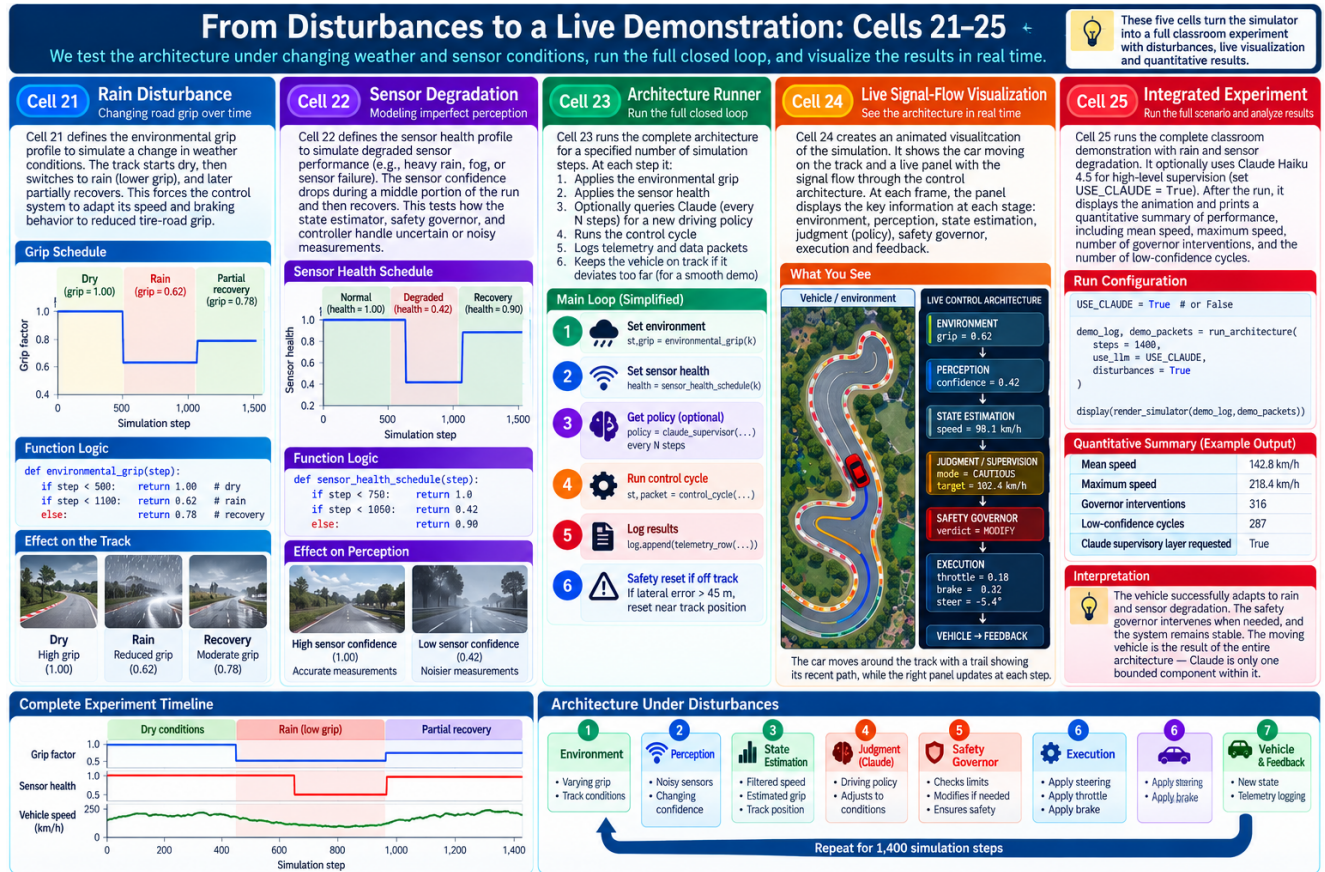


Figure 7: Cells 21–25: environmental disturbance, sensor degradation, architecture runner, live visualization and the integrated classroom experiment.

The disturbance schedule is structured to support causal comparison. The initial dry period provides a baseline in which grip and sensor health are both high. The rain transition then changes the physical environment while sensing remains healthy, allowing the estimator and controller to respond to a real reduction in capability. Later, sensor health declines while low grip persists. During this overlap, the architecture must manage both physical risk and uncertainty about its measurements. Recovery phases show whether behavior relaxes when conditions improve rather than remaining permanently conservative.

These regimes make several expected responses observable. Lower grip should reduce the planned safe speed and bring braking forward. Lower confidence should independently trigger restrictions through the governor, even if the measured trajectory has not yet deteriorated. When both occur together, the system should behave more cautiously than under either disturbance alone. If telemetry does not show these relationships, the architecture may contain a broken data path: for example, confidence might be computed but never consumed, or grip might be observed but excluded from planning.

The two-cadence runner is equally important. Vehicle motion, sensing and deterministic control execute at the fast simulation interval, while language-model supervision is called only periodically. This prevents API latency from defining the stability of the steering loop and reflects a general real-time design principle. Components should operate at the cadence appropriate to their function. Slow strategic judgment can update policy, but fast safety and control must remain available between those updates.

The pedagogical reset deserves careful interpretation. It is not evidence that a production vehicle may simply teleport back onto the track. It keeps a classroom demonstration informative after a large departure that would otherwise consume the remaining simulation. Because resets should be logged, they can still count as failures during analysis. The distinction between demonstration continuity and performance evaluation should remain explicit whenever the notebook is extended.

Visualization makes the temporal relationships legible. A viewer can connect rain to changing grip, sensor degradation to falling confidence, policy changes to supervisory calls, and command modifications to governor verdicts. The signal-flow panel discourages the mistaken impression that the moving marker is controlled by one monolithic model. It shows multiple transformations continuing even when the strategic policy remains unchanged.

The integrated experiment is therefore a system test rather than a showcase of isolated functions. It asks whether the interfaces remain coherent across 1,400 repeated cycles and across changes in both the world and the quality of

knowledge about the world. Summary statistics provide a compact assessment, while the telemetry retains cycle-level evidence for deeper diagnosis. A successful run is not merely one in which the car moves quickly. It is one in which speed adapts to grip, uncertainty causes appropriate caution, interventions are explainable, and recovery occurs without violating the designed authority boundaries.

The Architecture as a Hierarchy of Timescales and Authorities

The 25 cells can be read in two dimensions. The first is computational timescale. Vehicle dynamics, sensing, estimation and control operate every simulation step. Strategic supervision can operate less frequently because it changes policy rather than steering the car moment by moment. The second dimension is authority. Sensors can observe but cannot command. Estimators can infer but cannot actuate. The language model can recommend policy but cannot bypass validation. The governor can constrain commands. Execution alone changes the physical state.

Layer	Primary function	Typical timescale	Authority
Environment / dynamics	Define physical consequences	Continuous / every step	Physical causality
Perception	Measure the world	Fast	Observation only
State estimation	Construct internal state	Fast	Inference only
Deterministic planning/control	Compute local action	Fast	Proposal
Claude supervisor	Interpret context, set policy	Slower	Recommendation
Validation	Normalize semantic output	On supervisor call	Admissibility
Safety governor	Enforce hard constraints	Every step	Modify / constrain
Execution	Apply approved commands	Every step	Physical action
Telemetry	Preserve evidence	Every step	Observation / audit

This hierarchy is the deeper lesson of the notebook. A sophisticated autonomous system is not one intelligence operating everywhere. It is a federation of specialized intelligences and controls, coordinated by interfaces. Clear assignment of the dynamic driving task and its fallback responsibilities is also central to the terminology standardized by SAE for levels of driving automation (SAE International, 2021).

Timescale separation reduces unnecessary coupling. The dynamics integrator must advance frequently enough to approximate continuous motion. Sensors and estimators must refresh quickly enough for the controller to react before errors grow. Strategic policy, by contrast, should not oscillate with every noisy measurement. Updating it more slowly filters short-lived fluctuations and reserves contextual reasoning for changes that persist long enough to matter. The architecture remains responsive because the fast layers continue to operate under the most recently validated policy.

Authority separation answers a different question: who may cause what kind of change? A sensor may change the evidence available to the estimator, but not the steering angle. An estimator may change the internal belief about grip, but not the actuator directly. The supervisor may change a policy parameter, but only after validation. The tactical controller may propose commands, while the governor may reduce or override them. Execution alone crosses the boundary into physical action. This chain supplies a concrete answer to the governance problem often summarized as “human or model in the loop.” The more precise issue is which actor controls each transition.

The two hierarchies reinforce one another. Fast components generally receive narrower, more numerical responsibilities because their outputs must be timely and predictable. Slower components can integrate broader context because their outputs are mediated before execution. The safety governor is unusual and important: it operates at the fast cadence while holding high constraining authority. Its logic must therefore remain computationally light, deterministic and continuously available.

Interfaces make the federation testable. Every boundary can be evaluated with recorded or synthetic inputs. Perception packets can be replayed, policies can be fuzzed with malformed fields, controllers can be tested against extreme curvature, and the governor can be challenged with conflicting hazards. A monolithic end-to-end system may also be tested, but locating the source of a failure is harder when state, judgment and authority are entangled.

This structure supports graceful degradation. If strategic supervision is delayed, deterministic control can continue under a default or prior policy. If confidence drops, the governor can constrain behavior while the estimator continues working. If telemetry visualization fails, execution need not stop, although evidence collection should be treated as a serious operational requirement. Explicit dependencies reveal which failures demand immediate shutdown and which permit a restricted mode.

For institutional applications, the hierarchy prevents broad model capability from being mistaken for broad permission. A language model may analyze an exception persuasively without possessing the authority to approve a payment, alter a risk limit or transmit an order. Those permissions should be represented in code and organizational controls. The notebook’s timescales and authority levels therefore provide more than a software pattern: they offer a vocabulary

for assigning responsibility in any autonomous process that combines statistical inference, generated judgment, deterministic rules and consequential action.

Failure Modes and What the Notebook Makes Visible

Perception failure

Sensor degradation can increase measurement noise and lower confidence. The architecture responds by propagating confidence into downstream safety logic rather than pretending that the state estimate is certain.

Model judgment failure

A supervisor can return an unknown mode or extreme numerical values. Validation clips and normalizes them. Even a syntactically valid policy remains subordinate to the safety governor.

Physical-envelope violation

A tactical controller may request a target speed that is inconsistent with curvature and grip. The governor recomputes the physical envelope and reduces the command.

Track-boundary risk

Large lateral error is treated as a distinct hazard. The governor can cut throttle and impose strong braking even if the strategic policy is otherwise permissive.

Architectural opacity

A system can behave acceptably while remaining impossible to audit. The notebook addresses this by preserving packets and telemetry at each stage. The objective is not merely successful motion but inspectable causality.

The value of the notebook is that these failures appear at different boundaries and therefore invite different remedies. Increased sensor noise is not corrected by asking the strategic supervisor to produce more confident prose. An invalid policy field is not corrected by retuning the steering controller. Excessive target speed in a corner is not primarily a telemetry problem. Decomposition helps the researcher place each intervention where the corresponding assumption is owned.

Timing failure is another important category. A recommendation may be sensible when generated but stale when applied. If grip, confidence or lateral error changes substantially between supervisory calls, the fast loop and governor must respond using current estimates rather than waiting for a new policy. The notebook's slower model cadence makes this issue visible and demonstrates why real-time safety cannot depend on a remote generative response arriving on schedule.

Interface failure can occur even when every component behaves correctly in isolation. A field may be expressed in the wrong unit, use an unexpected name or be interpreted with an inverted sign. Because the notebook uses explicit packets and small schemas, such mistakes can be detected through assertions, replay tests and plots. Clear interfaces reduce the space in which silent semantic errors can survive.

Estimator failure deserves separate attention from sensor failure. Measurements may be noisy but unbiased while smoothing introduces lag, or the assumed update rule may respond too slowly to a grip transition. The telemetry should therefore preserve both measured and estimated quantities whenever possible. Comparing them with simulator ground truth during development reveals whether uncertainty originates in the sensor channel or the inference layer. In deployment, where ground truth is unavailable, consistency checks and redundant signals become even more important.

Control failure may appear as oscillation, persistent lateral bias, delayed braking or continual saturation at command limits. The governor can reduce the consequences but should not be used to conceal poor tuning. Repeated overrides are evidence that upstream proposals are systematically incompatible with constraints. Intervention rate is thus a diagnostic metric as well as a safety statistic.

Strategic failure can be subtler. A policy may be well formed and remain inside every numerical range while interpreting the situation poorly. The architecture limits the damage through bounded influence, slower updates and downstream governance, but evaluation is still required. Researchers can compare supervised runs with deterministic baselines, inspect rationales, and test scenario suites covering rain, recovery, low confidence and large path error. The correct question is not whether the model ever makes a mistake; it is whether the system contains, detects and learns from predictable classes of mistake.

Finally, evidence itself can fail through omitted fields, inconsistent timestamps or logs that record a proposal without the executed result. The notebook treats telemetry as part of the loop precisely because reconstruction cannot be improvised after an incident. Together, these visible failure modes turn the simulator into a laboratory for resilience. It teaches that reliability comes from anticipating where assumptions break and assigning each break a detection

signal, a bounded response and a record.

Why This Matters Beyond Autonomous Driving

The car is a concrete demonstration of a more general architecture for autonomous institutions. A financial system also receives imperfect signals, constructs state estimates, applies deterministic calculations, uses models for contextual judgment, validates outputs, enforces limits, executes permitted actions and records evidence. The mapping is structural rather than literal.

Autonomous vehicle	Autonomous financial institution
Sensors	Market data, ledgers, documents, client inputs
State estimator	Risk state, liquidity state, exposure state
Curvature / grip constraints	Capital, liquidity, limits, legal constraints
Tactical controller	Pricing, hedging, routing, portfolio logic
LLM supervisor	Contextual judgment, exception analysis, strategic synthesis
Safety governor	Risk limits, compliance gates, approval matrices
Actuators	Orders, payments, approvals, communications
Telemetry	Audit trail, model logs, control evidence

The analogy clarifies why “put an LLM in the loop” is not an architecture. The surrounding system determines what the model sees, what it may change, how often it speaks, what numerical range is acceptable, who can override it, and what evidence survives after action.

In finance, the environment is partly physical, partly informational and partly institutional. Market prices change independently of the system, but accounting conventions, settlement calendars and regulatory rules also define the state in which action occurs. A useful architecture must represent these conditions explicitly. Just as track curvature cannot be inferred from eloquent language alone, capital availability or payment eligibility should be calculated from authoritative data and deterministic rules.

Perception corresponds to acquiring and normalizing evidence from feeds, documents, ledgers and counterparties. Those sources have different latencies and failure modes. A price may be stale, a document may be ambiguous, a ledger entry may be pending, and a client instruction may require authentication. Confidence and provenance therefore matter as much as the observed value. A downstream model should know whether a claim came from a signed system of record or an uncertain extraction from unstructured text.

State estimation creates a coherent operational picture from that evidence. Risk, liquidity and exposure are not single sensor readings; they are constructed views with assumptions, aggregation rules and update schedules. Separating those views from action permits reconciliation and challenge before a trade or payment is proposed. It also allows multiple decision tools to consume the same governed state rather than each inventing an incompatible representation.

Deterministic controllers correspond to quantitative routines whose objectives and limits can be expressed precisely: pricing formulas, hedge ratios, routing rules, cash ladders or portfolio constraints. A language model can contribute by interpreting a novel event, summarizing documentation, identifying an exception or proposing a strategic posture. Its recommendation should still enter through a typed interface. For example, it might recommend a more conservative liquidity buffer within a permitted interval rather than directly issuing transfers.

The safety governor maps naturally to pre-trade controls, compliance checks, segregation of duties, approval matrices and hard exposure limits. These controls should evaluate the current state and the proposed action, not merely the quality of the model’s explanation. A persuasive rationale cannot authorize a prohibited transaction. Conversely, a valid proposal may require modification because the institution’s position changed after it was generated.

Execution is the point at which messages become orders, payments, approvals or client communications. That boundary should be narrow, authenticated and idempotent where possible. Telemetry must preserve the evidence used, the recommendation made, every validation or override, the authorized action and the external response. This causal record supports operations, audit, model risk management and accountability.

The analogy has limits: a financial institution contains people, legal duties and adversarial participants that exceed the toy physics of a race-car simulator. Nevertheless, the structural lesson remains powerful. Dependable autonomy requires a shared state, specialized reasoning, bounded permissions, independent constraints and durable evidence. A capable language model can improve judgment within that arrangement, but capability alone neither defines authority nor discharges institutional responsibility.

Design Principles Emerging from the Simulator

Separate perception from judgment

Measurements are not conclusions. Preserve provenance, confidence and uncertainty before higher-level reasoning begins.

Separate state from action

A stable internal representation reduces the temptation to react directly to noisy inputs. Estimation is an architectural layer in its own right.

Use deterministic intelligence where invariants dominate

Geometry, clipping, target-speed calculations and safety checks are more naturally expressed as deterministic functions than as prose reasoning.

Give generative intelligence bounded strategic authority

The language model is most useful as a slower contextual supervisor. It changes policy parameters rather than controlling actuators directly.

Validate before governing; govern before executing

These are distinct controls. Validation checks whether a recommendation has an admissible shape and range. Governance checks whether an otherwise valid recommendation is safe in the current state.

Make uncertainty actionable

Confidence should influence behavior. A system that knows its sensors are unreliable should become more conservative.

Log the causal chain

Telemetry should preserve not only outcomes but also intermediate beliefs, proposals, interventions and executed commands. This makes accountability architectural rather than retrospective.

These principles are most useful when treated as design tests rather than slogans. For separation of perception and judgment, a reviewer should be able to identify the source, timestamp, unit and confidence of every material input. For separation of state and action, it should be possible to replay a recorded state into the decision layer without reconnecting live sensors. For deterministic invariants, every hard limit should have executable logic and boundary tests. The architecture passes only when these properties are visible in interfaces and code.

Bounded strategic authority requires both a narrow action vocabulary and a clear default. If the generative component times out, returns malformed output or becomes unavailable, the system should know whether to retain the last safe policy, enter a cautious mode or stop. That choice depends on the application, but it must be made before failure. In the notebook, the deterministic loop supplies a baseline capability and the policy schema constrains supervisory influence.

Validation and governance should also produce different evidence. A validation log answers whether a message conformed to schema, type and range. A governance log answers whether an admissible proposal was safe and permitted in the current context. Combining the two into a single “accepted” flag would obscure whether the problem lay in communication or judgment. Separate verdicts support more precise monitoring and remediation.

Making uncertainty actionable means defining behavior at confidence thresholds. It is insufficient to display a confidence score if no component consumes it. Designers should specify which privileges shrink, which limits tighten and which escalation occurs as evidence quality deteriorates. The response should be proportionate: a modest reduction in confidence may justify slower operation, while a severe or contradictory signal may require an emergency posture.

Causal logging must be designed with identity and time in mind. State, policy, proposal, governor verdict and execution should share a cycle identifier so they can be joined reliably. Strategic recommendations should include the conditions under which they were generated and the interval for which they remain active. Logs should record both raw and normalized model output when policy permits, because normalization can materially change meaning. In sensitive domains, the same design must also address retention, access control and privacy.

An additional principle follows from the experiment: evaluate recovery, not only degradation. Safe systems should become conservative when rain or sensor uncertainty appears, but they should also return toward normal operation

when trustworthy evidence supports recovery. Permanent caution after a transient fault can be operationally costly and may indicate that state or policy has become sticky. Scenario tests should therefore cover onset, overlap and resolution of disturbances.

Finally, architecture should make replacement possible. A different estimator, controller or language model should be insertable behind the same contract and evaluated against the same scenarios. Replaceability prevents one implementation from becoming synonymous with the system's purpose. It also supports progressive improvement while preserving the surrounding safety case. The simulator embodies these principles in compact form, giving researchers a concrete checklist for larger autonomous systems.

Conclusion

The notebook demonstrates autonomous intelligence by refusing to treat intelligence as a single object. The car moves because many components cooperate under different informational and authority constraints. The environment supplies conditions. Sensors generate observations. State estimation constructs a usable internal world. Planning translates curvature and grip into a target. Deterministic controllers produce fast local commands. Claude Haiku 4.5 contributes slower strategic judgment. Validation constrains the semantic output. The safety governor enforces non-negotiable limits. Execution changes the physical state. Telemetry preserves evidence. Feedback begins the cycle again.

The resulting system is more intelligible precisely because no component is asked to do everything. The LLM is neither minimized nor mythologized. It occupies a role for which language-model intelligence is well suited: contextual interpretation and strategic adjustment. But the architecture retains deterministic computation, physical constraints and explicit governance where those mechanisms are stronger.

FINAL TAKEAWAY

The autonomous car is not “Claude driving.” It is a governed control architecture in which Claude is one bounded source of judgment. The same principle generalizes: the path from a capable model to a dependable autonomous institution runs through architecture – perception, state, tools, rules, validation, authority, execution, feedback and evidence.

The experiment’s most important contribution is explanatory discipline. It assigns each claim to a mechanism that can be inspected. When the car slows in rain, the reader can follow the change from environmental grip through sensing, estimation, target-speed calculation, policy and governance. When sensor health declines, confidence becomes visible and changes the permitted command. When the supervisor recommends a policy, its output is recorded before and after normalization. The demonstration therefore explains behavior through a chain of transformations rather than through anthropomorphic shorthand.

That discipline supports responsible experimentation. Researchers can disable Claude supervision and establish a deterministic baseline, change the disturbance schedule, alter estimator smoothing, challenge validation with malformed outputs or strengthen governor thresholds. Each variation tests a particular architectural hypothesis. Results can be compared using telemetry rather than impressions from an animation. Because the notebook is executable and the paper identifies the role of every cell, conceptual arguments remain connected to code.

The architecture also clarifies that autonomy and control are not opposites. An autonomous system can act without continuous human instruction while remaining subject to designed limits, escalation rules and evidence requirements. Indeed, consequential autonomy becomes more credible when permissions are narrow and overrides are explicit. The governor does not diminish the intelligence of the overall system; it converts broad capability into dependable agency.

No claim is made that the simplified vehicle, sensor model or circuit is production ready. Real autonomous driving requires richer dynamics, perception from high-dimensional sensors, localization, prediction of other actors, fault-tolerant hardware, formal verification and extensive safety validation. Functional safety, safety of the intended functionality and structured safety assessment are addressed by substantially richer standards and guidance than this teaching simulator attempts to implement (International Organization for Standardization, 2018, 2022; National Highway Traffic Safety Administration, 2017). The value of simplification is that it exposes relationships that a production stack can hide. A reader can see why ground truth differs from measurement, why an estimate differs from an action, and why a valid recommendation can still be unsafe.

The transfer to finance and other institutional settings should likewise preserve the structure rather than the literal details. Curvature becomes a domain constraint, grip becomes available operating capacity, sensor health becomes data confidence, and actuation becomes a consequential transaction. The essential questions remain: What is known? How certain is it? Who may recommend? Who may authorize? Which rules cannot be overridden? What exactly was executed? What evidence permits reconstruction?

The companion notebook makes those questions runnable. Its 25 cells provide a compact reference implementation in which strategic language-model judgment has a genuine role without being confused with the whole of intelligence. The result is a more mature account of autonomous systems: not a solitary model producing commands, but a recurrent, governed process that combines multiple forms of computation. Dependability emerges from their coordination, their limits and the feedback that allows the architecture to observe the consequences of its own actions.

Appendix: Cell Map and Reproducibility

Cell-by-Cell Architectural Map

Cell	Role	Architectural contribution
1	Install/import	Establishes the simulation and Anthropic software stack.
2	Configuration	Fixes seed, time step and vehicle limits for reproducibility.
3	Circuit geometry	Creates the simplified closed teaching circuit.
4	Reference frame	Computes distance, heading and curvature along the track.
5	Vehicle state	Defines the physical state carried through the simulation.
6	Dynamics	Maps governed commands into next physical state.
7	Ideal sensors	Defines perfect observations of track and vehicle context.
8	Noise/confidence	Converts perfect observation into uncertain measurement.
9	Perception packet	Structures sensor outputs for downstream components.
10	State estimator	Smooths measurements and constructs an internal state.
11	Braking demand	Converts curvature and grip into a safe-speed feature.
12	Lateral controller	Produces deterministic steering.
13	Longitudinal controller	Produces deterministic throttle/brake.
14	Tactical proposal	Combines planning, policy and fast control.
15	Claude supervisor	Recommends high-level policy from a compact summary.
16	Validation	Normalizes mode, ranges, Boolean values and rationale.
17	Safety governor	Enforces physical, confidence, boundary and emergency constraints.
18	Execution	Applies governed commands to the vehicle model.
19	Closed loop	Orchestrates perceive-estimate-propose-govern-execute.
20	Telemetry	Records state, policy, commands and governor verdict.
21	Rain disturbance	Changes environmental grip over time.
22	Sensor disturbance	Changes sensor health over time.
23	Runner	Repeats the architecture and schedules slower supervision.
24	Renderer	Visualizes motion and live signal flow.
25	Experiment	Runs the integrated 1,400-step classroom demonstration.

The map is intended to be read as an authority trace as well as a contents list. Cells 1–8 define the world and the evidence available about it. Cells 9–14 transform evidence into an estimated state and a tactical proposal. Cells 15–16 add strategic model judgment but constrain its output contract. Cells 17–20 determine what may be executed and preserve the resulting evidence. Cells 21–25 expose the complete architecture to changing physical and informational conditions. Reading vertically through the table therefore reveals where information is created, where it is interpreted and where permission to affect the simulated vehicle is finally exercised.

Source and Reproducibility Note

This paper is derived from the companion notebook `TOPIC_783_SELF_DRIVING_RACE_CAR.ipynb` and the seven supplied pedagogical infographics. The paper follows the architecture and visual language of the supplied LaTeX reference document. The notebook itself remains the executable source of truth for implementation details and numerical outputs; summary values shown in illustrative graphics should be interpreted as examples unless reproduced by running the notebook in the intended Colab environment. The notebook can be opened directly through the following persistent document link: [Google Colab companion notebook](#).

For reproducible use, a reader should execute the cells in numerical order, retain the configured random seed and record whether external Claude supervision is enabled. Runs that use a live model may vary because service responses and availability are external to the deterministic simulator. The validated policy, governor verdict and executed command should therefore be retained with the telemetry whenever results are compared. A deterministic baseline run without the external supervisor provides a useful reference for separating changes caused by policy judgment from changes caused by the vehicle, disturbance schedule or controller.

The architectural descriptions in this paper document authority boundaries rather than replacing the notebook. Readers extending the experiment should preserve these boundaries even when replacing individual algorithms. In particular, generated recommendations should remain typed and bounded, and the control-cycle ordering should continue to prevent a proposal from reaching dynamics before governance. Versioning the notebook, documenting modified constants and exporting telemetry with each reported experiment will keep claims in the paper traceable to executable evidence.

References

- Anthropic. (2025). *Claude Haiku 4.5 System Card*. <https://assets.anthropic.com/m/99128ddd009bdcb/original/Claude-Haiku-4-5-System-Card.pdf>.
- International Organization for Standardization. (2018). *ISO 26262: Road Vehicles—Functional Safety* (second edition). Geneva: ISO. <https://www.iso.org/publication/PUB200262.html>.
- International Organization for Standardization. (2022). *ISO 21448:2022 Road Vehicles—Safety of the Intended Functionality*. Geneva: ISO. <https://www.iso.org/standard/77490.html>.
- Kalman, R. E. (1960). A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1), 35–45. <https://doi.org/10.1115/1.3662552>.
- National Highway Traffic Safety Administration. (2017). *Automated Driving Systems 2.0: A Vision for Safety*. Washington, DC: U.S. Department of Transportation. <https://www.nhtsa.gov/vehicle-manufacturers/automated-driving-systems>.
- National Institute of Standards and Technology. (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1. <https://doi.org/10.6028/NIST.AI.100-1>.
- Paden, B., Čáp, M., Yong, S. Z., Yershov, D., and Frazzoli, E. (2016). A survey of motion planning and control techniques for self-driving urban vehicles. *IEEE Transactions on Intelligent Vehicles*, 1(1), 33–55. <https://doi.org/10.1109/TIV.2016.2578706>.
- Pendleton, S. D., Andersen, H., Du, X., Shen, X., Meghjani, M., Eng, Y. H., Rus, D., and Ang, M. H. (2017). Perception, planning, control, and coordination for autonomous vehicles. *Machines*, 5(1), 6. <https://doi.org/10.3390/machines5010006>.
- Rajamani, R. (2012). *Vehicle Dynamics and Control* (2nd ed.). New York: Springer. <https://doi.org/10.1007/978-1-4614-1433-9>.
- SAE International. (2021). *J3016_202104: Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles*. https://doi.org/10.4271/J3016_202104.
- Seto, D., Krogh, B. H., Sha, L., and Chutinan, A. (1998). Dynamic control system upgrade using the Simplex architecture. *IEEE Control Systems*, 18(4), 72–80. <https://doi.org/10.1109/37.710880>.
- Thrun, S., Montemerlo, M., Dahlkamp, H., et al. (2006). Stanley: The robot that won the DARPA Grand Challenge. *Journal of Field Robotics*, 23(9), 661–692. <https://doi.org/10.1002/rob.20147>.
- Urmson, C., Anhalt, J., Bagnell, D., et al. (2008). Autonomous driving in urban environments: Boss and the Urban Challenge. *Journal of Field Robotics*, 25(8), 425–466. <https://doi.org/10.1002/rob.20255>.